Cloud Computing & Distributed Systems

Comprehensive Revision Notes

Topics Covered:

SaaS · PaaS · IaaS · FaaS · Virtualization
Distributed Systems · FTP · SNTP · XaaS
Grid · Cluster · Cloud Computing
Full & Para Virtualization · Community Cloud
DOM · SAX · StAX · SMS · HTTP · SMTP · UDDI
Service-Oriented Architecture (SOA)

April 17, 2026

Contents

Cloud Service Models: SaaS, PaaS, IaaS & FaaS

What are Cloud Service Models?

Cloud service models define **how much responsibility** the cloud provider takes versus how much the user manages. Think of it as a spectrum from "everything done for you" to "you control everything."

1.1 SaaS — Software as a Service

▶ What is it?

You simply **use** the software. No setup, no coding, no installation worries. The provider manages absolutely everything.

Think of it as a **ready-made app** that you just open and start using.

⋆ Key Points to Remember

- You manage: **Nothing**
- Provider manages: **Everything** (servers, OS, runtime, application)
- Accessible via browser or app
- Subscription-based pricing

Example: SaaS in Action

- **Google Docs** — Write documents online without installing Microsoft Word
- **Netflix** — Watch movies without owning or managing any server
- **Gmail** — Send emails without setting up a mail server

★ Quick Tip / Memory Trick

SaaS = Just open and use. Like watching TV — you don't worry about the broadcast tower.

1.2 PaaS — Platform as a Service

▶ What is it?

You **write code** and the platform handles deployment, servers, OS, and runtime. It gives developers an environment to build and deploy applications.

Think of it as an **environment for developers** — you focus on coding, not infrastructure.

⋆ Key Points to Remember

- You manage: **Code only**
- Provider manages: **Servers, OS, runtime, middleware**
- Ideal for developers building web apps
- Speeds up development lifecycle

Example: PaaS in Action

- **Heroku** — Deploy your app with a single command; no server management
- **Google App Engine** — Host web applications on Google's infrastructure
- **Firebase** — Build mobile/web apps with backend provided

★ Quick Tip / Memory Trick

PaaS = Focus on coding, not infrastructure. Like renting a kitchen to cook — you bring the recipe, not the kitchen equipment.

1.3 IaaS — Infrastructure as a Service

▶ What is it?

You get **virtual machines** and control everything above the hardware level. Maximum control, but also maximum responsibility.

Think of it as **renting hardware online** — you get the raw machine and configure it yourself.

⋆ Key Points to Remember

- You manage: **OS, applications, runtime, middleware**
- Provider manages: **Hardware, networking, physical data center**
- Most flexible cloud model
- Best for DevOps and system administrators

Example: IaaS in Action

- **Amazon EC2** — Rent virtual servers and configure them completely
- **Microsoft Azure Virtual Machines** — Get VMs on Azure cloud
- **Google Compute Engine** — Google's virtual machine infrastructure

★ Quick Tip / Memory Trick

IaaS = Maximum control, more responsibility. Like renting an empty apartment — you furnish and manage it yourself.

1.4 FaaS — Function as a Service (Serverless)

▶ What is it?

You run **small pieces of code (functions)** only when needed. You don't think about servers at all — the provider manages everything. This is called **serverless computing**.

⋆ Key Points to Remember

- You manage: **Function code only**
- Provider manages: **Everything else** (scaling, servers, OS)
- Runs **only when triggered** — no idle cost
- Pay only for execution time (very cost-efficient)
- Perfect for event-driven applications

Example: FaaS in Action

- **AWS Lambda** — Run code when a file is uploaded to S3
- **Google Cloud Functions** — Trigger code on HTTP request
- **Azure Functions** — Event-driven serverless compute

★ Quick Tip / Memory Trick

FaaS = Runs only when triggered → cost efficient. Like paying electricity only when the light is ON.

1.5 Quick Comparison Table

Feature	SaaS	PaaS	IaaS	FaaS
Control	None	Medium	Full	Very Low
You Manage	Nothing	Code	OS + Apps	Function
Use Case	End Users	Developers	DevOps	Event Apps
Example	Gmail	Heroku	AWS EC2	AWS Lambda
Setup Needed	No	Little	High	Very Little
Cost Model	Subscription	Usage	Usage	Per execution

★ Quick Tip / Memory Trick

Easy Way to Remember:
- **SaaS** → *Use* app
- **PaaS** → *Build* app

- **IaaS** → *Control* servers
- **FaaS** → *Run* functions

Virtualization — Core Concepts

What is Virtualization?

Virtualization is the process of creating **virtual (simulated) versions** of physical resources — such as servers, operating systems, storage devices, or networks — using software, so that one physical machine can behave like many independent machines.

2.1 How Virtualization Works

The core idea is: **One physical computer → Many virtual machines (VMs)**

A software layer called the **Hypervisor** sits between the physical hardware and the virtual machines. It allocates resources (CPU, RAM, storage) to each VM and manages them independently.

✓ Real-Life Analogy

Think of a **hostel building**:
- **Building** = Physical Machine
- **Rooms** = Virtual Machines
- **Warden** = Hypervisor
- Each student (VM) has their own room, uses shared electricity and water, and doesn't disturb others.

Example: Your VirtualBox Setup

- **Windows** (your real OS) = Host OS
- **Oracle VirtualBox** = Hypervisor (Type 2)
- **Kali Linux** (inside VirtualBox) = Guest OS / Virtual Machine

VirtualBox creates a *fake computer* (VM) and gives Kali Linux virtual CPU, virtual RAM, and a virtual disk. Kali Linux thinks it is running on a real machine!

2.2 The Hypervisor — Brain of Virtualization

The hypervisor is the most critical component. It comes in two types:

	Type 1 (Bare Metal)	Type 2 (Hosted)
Runs On	Directly on hardware	On top of host OS
Performance	Faster, more efficient	Slightly slower
Used For	Enterprise / data centres	Personal / testing
Examples	VMware ESXi, Hyper-V	VirtualBox, VMware Workstation

2.3 Key Points to Remember

⋆ Key Points to Remember

1. **One → Many:** One physical system can host many virtual machines simultaneously.
2. **Hypervisor is the Brain:** It manages all VMs, allocates resources, and controls execution.
3. **Isolation:** Each VM is completely independent — if one crashes, others remain unaffected.
4. **Resource Sharing:** CPU, RAM, and storage are shared among VMs. Example: 16 GB RAM laptop can give 4 VMs with 4 GB each.
5. **Platform Independence:** Run Linux on a Windows machine, or vice versa.
6. **Cost Efficient:** No need to buy multiple physical servers — saves hardware and electricity costs.
7. **Scalability:** Easily create or delete VMs on demand — highly useful in cloud computing.
8. **Security Advantage:** Isolation means a virus in one VM does not spread to others.

2.4 Types of Virtualization

Type	Description
Server Virtualization	One physical server → multiple virtual servers
OS Virtualization	Multiple operating systems running on one physical machine
Storage Virtualization	Combine multiple physical storage devices into one logical unit
Network Virtualization	Create virtual networks on top of physical network infrastructure

2.5 Full Virtualization vs Para Virtualization

Full Virtualization

The guest OS runs **without any modification**. The OS does **not know** it is running in a virtual environment. The hypervisor creates completely fake hardware, and the guest OS thinks it is on a real machine.

Example: Running Kali Linux in VirtualBox without modifying Kali.

Para Virtualization

The guest OS is **modified (aware)** that it runs inside a virtual environment. It communicates directly with the hypervisor for better performance. The OS is adapted to call the hypervisor instead of hardware directly.

Example: Xen hypervisor running a modified Linux kernel.

Feature	Full Virtualization	Para Virtualization
OS Modification	Not required	Required
OS Awareness	OS is unaware	OS knows it is virtual
Performance	Slightly slower	Faster
Complexity	Easier to set up	More complex
Example	VirtualBox + Kali	Xen + Modified Linux

★ Quick Tip / Memory Trick

Memory Trick:
- **Full Virtualization** → "OS is fooled" (doesn't know it's virtual)
- **Para Virtualization** → "OS knows the truth" (cooperates with hypervisor)

Your VirtualBox + Kali Linux setup = Full Virtualization + Type 2 Hypervisor

• Exam-Ready Definition

Virtualization is the process of creating virtual versions of physical resources (servers, OS, storage) using a **hypervisor**, allowing multiple independent virtual machines to run simultaneously on a single physical machine — improving efficiency, reducing cost, and enabling scalability.

Distributed Systems

What is a Distributed System?

A **distributed system** is a collection of **independent computers** that communicate over a network and work together to appear as a **single unified system** to the end user.

3.1 Core Concept

To the user, it **feels like one system**. Internally, many machines are cooperating, sharing the workload.

Example: Instagram / Google Drive

When you upload a file to Google Drive:
- It is *not* stored on one computer
- It is distributed across many servers in different geographical locations
- Yet you feel like you are using just one simple app

✓ Real-Life Analogy

Think of a **food delivery system**:
- **Many restaurants** (servers) = Different nodes
- **Many delivery agents** (nodes) = Computing units
- **One app** (system) = Single interface the user sees
- Even if one restaurant closes → others still work!

3.2 Key Properties of Distributed Systems

⋆ Key Points to Remember

1. **Multiple Systems Work Together:** Many computers collaborate to act as one.
2. **Transparency:** The user doesn't know it's distributed — it looks like a single system.
3. **Resource Sharing:** Data, files, and services are shared across nodes.
4. **Scalability:** Easily add more machines without stopping the system.
5. **Fault Tolerance:** If one machine fails, others continue — no complete failure.
6. **Concurrency:** Multiple users can access the system simultaneously.
7. **Communication via Network:** Nodes communicate using message passing over a network.
8. **No Global Clock:** Each node has its own clock — time synchronisation is difficult.

3.3 Types of Distributed Systems

Type	Description	Example
Client-Server	One server serves many clients	Gmail, Web Servers
Peer-to-Peer (P2P)	All nodes are equal — no central server	BitTorrent
Cloud-Based	Services delivered over the internet	AWS, Azure

3.4 Advantages and Disadvantages

Advantages:

✓ High performance
✓ Fault tolerance
✓ Scalability
✓ Resource sharing
✓ Geographic distribution

Disadvantages:

× Complex design
× Security issues
× Network dependency
× Debugging is very difficult
× Synchronisation challenges

★ **Quick Tip / Memory Trick**

Key Distinction:
- **Virtualization** → 1 machine → many virtual machines
- **Distributed System** → many machines → 1 unified system

• **Exam-Ready Definition**

A **distributed system** is a collection of independent computers that communicate over a network and work together as a single system, providing transparency, fault tolerance, and scalability to achieve a common goal.

Grid, Cluster, and Cloud Computing

This is an important trio — let's understand each clearly and not confuse them.

4.1 Cluster Computing

Cluster Computing

A **cluster** is a group of computers (*nodes*) that are **physically located at the same place**, connected via a high-speed network, and work together as one powerful system.

⋆ Key Points to Remember

- Systems are in the **same location** (same data centre or lab)
- Connected with **high-speed, low-latency** network
- Works like a single **very powerful computer**
- Used for **high-performance computing (HPC)** tasks
- **Tightly coupled** — nodes depend on each other
- Control is **centralised**

Example: Cluster in Action

- **Supercomputers** used for complex scientific simulations
- **Weather prediction systems** processing massive data
- High-performance rendering farms for animation studios

✓ Real-Life Analogy

Group of students solving one assignment **together in one room**. All are in the same place, working on the same problem, side by side.

4.2 Grid Computing

Grid Computing

Grid computing involves multiple computers from **different geographical locations** working together, sharing resources over the internet to solve a common large-scale problem.

⋆ Key Points to Remember

- Systems are **geographically distributed** (different cities/countries)
- Each system works **independently** and contributes spare resources
- Resources (CPU, storage) shared **over the internet**
- **Loosely coupled** — nodes are relatively independent

- Goal is **resource sharing**, not just performance

Example: Grid in Action

- **SETI@home** — Thousands of home computers worldwide analyse radio signals from space to search for extraterrestrial intelligence
- Scientific research projects using computing power donated by volunteers

✓ Real-Life Analogy

Students in different cities collaborating online. Each is at their own location, contributing to a shared project over the internet.

4.3 Cloud Computing

Cloud Computing

Cloud computing is the delivery of computing services — servers, storage, databases, networking, software — **over the internet ("the cloud")** on demand, with a pay-as-you-use pricing model.

⋆ Key Points to Remember

- Resources available **on demand** — get what you need instantly
- **Pay-as-you-go** — only pay for what you use
- Accessible from **anywhere** via internet
- Built using both **distributed computing** and **virtualization**
- **Provider-managed** infrastructure
- Highly **scalable** and **elastic**

Example: Cloud in Action

- **Google Drive** — Store files and access from any device
- **Amazon Web Services (AWS)** — Rent servers, databases, AI tools
- **Microsoft Azure, Google Cloud Platform**

✓ Real-Life Analogy

Like using **electricity or water from the utility company**. You don't build your own power plant — you rent what you need, pay for usage, and access it anywhere.

4.4 Cluster vs Grid vs Cloud — Full Comparison

Feature	Cluster	Grid	Cloud
Location	Same place	Different locations	Anywhere (internet)
Connection	Tight (high-speed LAN)	Loose (internet)	Internet-based
Goal	High performance	Resource sharing	Service delivery
Control	Centralised	Distributed	Provider-managed
Coupling	Tightly coupled	Loosely coupled	On-demand
Example	Supercomputer	SETI@home	AWS, Google Drive
Scaling	Fixed nodes	Add volunteers	Instant / elastic

★ Quick Tip / Memory Trick

Easy Memory Trick:
- **Cluster** → Together, *same place*, high performance
- **Grid** → Separate, *different places*, resource sharing
- **Cloud** → *Service via internet*, pay-as-you-go

If confused in exam: Same location → Cluster, Different locations → Grid, Service over internet → Cloud

Community Cloud

What is Community Cloud?

A **Community Cloud** is a cloud infrastructure that is **shared by multiple organisations** that have similar requirements, goals, security concerns, or compliance needs.

In one line: "Same type of organisations share one cloud."

5.1 Key Features

⋆ Key Points to Remember

- Shared by a **specific community** of organisations (not the general public)
- All members follow the **same security and compliance rules**
- **Cost is shared** among member organisations
- **More secure** than public cloud (restricted access)
- **Less expensive** than private cloud (shared infrastructure)
- Jointly managed by the community or a third-party provider

5.2 Real-World Examples

Example: Healthcare

All hospitals in a region share a community cloud to:
- Store patient records securely
- Follow the same privacy regulations (e.g., HIPAA)
- Share medical research data

Example: Government

Different government departments share a community cloud for:
- Citizen data management
- Inter-department data sharing
- High-security data governance

Example: Education

Universities share a community cloud for:
- Research collaboration
- Student data management
- Online learning platforms

5.3 Four Cloud Deployment Models — Comparison

Cloud Type	Who Uses It	Cost	Example
Public Cloud	Anyone	Lowest	Amazon Web Services
Private Cloud	Single organisation	Highest	Company internal cloud
Community Cloud	Similar organisations	Medium (shared)	Hospitals, Government
Hybrid Cloud	Mix of public + private	Varies	Company + AWS

✓ **Real-Life Analogy**

- **Public Cloud** = Public bus (anyone can board)
- **Private Cloud** = Personal car (only you use it)
- **Community Cloud** = Office cab shared by employees of same company
- **Hybrid Cloud** = You sometimes take the cab, sometimes drive your car

• **Exam-Ready Definition**

A **Community Cloud** is a shared cloud infrastructure used exclusively by a group of organisations with common goals, compliance requirements, or security needs, where the cost and resources are shared among the members.

Protocols and Standards: FTP, SNTP, XaaS, FaaS

6.1 FTP — File Transfer Protocol

FTP — File Transfer Protocol

FTP is a standard network protocol used to **transfer files between computers** over a TCP/IP network (the internet or LAN). It follows the client-server model.

⋆ Key Points to Remember

- Works on **client-server model**
- Uses **Port 21** (control channel) and Port 20 (data channel)
- Supports both **upload and download** of files
- Requires **username and password** for authentication
- **FTPS** = FTP + SSL encryption (secure version)
- **SFTP** = SSH File Transfer Protocol (even more secure)

Example: FTP in Action

A web developer uses **FileZilla** (FTP client) to upload website files from their laptop to a web server. They enter the server's IP, username, and password — then drag and drop files to transfer them.

6.2 SNTP — Simple Network Time Protocol

SNTP — Simple Network Time Protocol

SNTP is a simplified version of **NTP (Network Time Protocol)** used to **synchronise the clocks** of computers across a network by communicating with time servers.

⋆ Key Points to Remember

- Simplified, lightweight version of NTP
- Keeps system clocks **accurate and synchronised**
- Critical in **distributed systems** (timestamps must match)
- Operates using **UDP on Port 123**
- Used in devices where full NTP is too resource-heavy

Example: SNTP in Action

Your smartphone **automatically syncs its time** from an internet time server whenever connected to the internet. This ensures your clock is always accurate without you manually setting it.

6.3 XaaS — Anything as a Service

XaaS — Anything as a Service

XaaS is an umbrella term for **all types of cloud services**. The "X" stands for anything — it encompasses SaaS, PaaS, IaaS, FaaS, and every other "as a Service" model.

⋆ Key Points to Remember

- **"X" = Anything** — the term is intentionally broad
- Represents the shift from **owning** software/hardware to **renting** it as a service
- Includes: SaaS, PaaS, IaaS, FaaS, DBaaS (Database), NaaS (Network), etc.
- Everything is **delivered over the internet** on demand

Example: XaaS in Action

- **Google Drive** (storage as a service)
- **Amazon Web Services** (infrastructure + platform + software as a service)
- **Office 365** (productivity software as a service)

6.4 Quick Comparison Table

Term	Full Form	Use / Purpose
FTP	File Transfer Protocol	Transfer files between computers
SNTP	Simple Network Time Protocol	Synchronise system clocks
XaaS	Anything as a Service	General term for all cloud services
FaaS	Function as a Service	Run code functions without managing servers

★ Quick Tip / Memory Trick

Super Short Exam Definitions:

- **FTP:** Protocol used to transfer files over a network (Port 21)
- **SNTP:** Protocol used to synchronise system clocks over a network
- **XaaS:** Model where everything (software, hardware, functions) is provided as a service over the internet
- **FaaS:** Cloud service model to run functions/code without managing server infrastructure

Communication Protocols: SMS, HTTP, SMTP, UDDI

7.1 SMS — Short Message Service

SMS — Short Message Service

SMS is a service that enables **short text messages** (up to 160 characters) to be exchanged between mobile phones using the mobile telephone network.

⋆ Key Points to Remember

- Works on **mobile cellular network** (no internet needed)
- Limit of **160 characters** per message
- Fast, simple, and widely supported
- Used for **OTPs**, alerts, notifications, and marketing

Example: SMS in Action

When you log in to your bank account, a **One-Time Password (OTP)** like "Your OTP is 123456" is sent via SMS to verify your identity. No internet required on your phone.

7.2 HTTP — HyperText Transfer Protocol

HTTP — HyperText Transfer Protocol

HTTP is the foundational protocol for data communication on the **World Wide Web**. It defines how browsers request web pages and how servers respond with content.

⋆ Key Points to Remember

- Follows a **Request–Response** model
- **Stateless** — each request is independent (server doesn't remember previous requests)
- Uses **Port 80** (HTTP) and **Port 443** (HTTPS — secure version)
- **HTTPS** = HTTP + TLS/SSL encryption
- Methods: GET (retrieve), POST (send), PUT (update), DELETE (remove)

Example: HTTP in Action

When you open **www.google.com**, your browser sends an HTTP GET request. Google's server processes it and sends back the webpage HTML. Your browser then renders the page you see.

7.3 SMTP — Simple Mail Transfer Protocol

SMTP — Simple Mail Transfer Protocol

SMTP is the standard protocol for **sending emails** from a client to a mail server, and from one mail server to another.

⋆ Key Points to Remember

- Used **only for sending** emails (not receiving)
- Email **receiving** uses POP3 (Port 110) or IMAP (Port 143)
- Works on **Port 25** (or 587 for submission)
- Operates between mail servers in a **store-and-forward** manner
- Gmail, Yahoo, Outlook all use SMTP to send emails

Example: SMTP in Action

When you click **Send** in Gmail, your email goes to Google's SMTP server. That server then forwards it to the recipient's mail server. The recipient's mail client (using IMAP) downloads it.

7.4 UDDI — Universal Description, Discovery and Integration

UDDI — Universal Description

UDDI is a **platform-independent, XML-based registry** that allows businesses to **publish, discover, and integrate** web services. It acts like a "yellow pages" directory for web services.

⋆ Key Points to Remember

- Acts as a **service registry / directory**
- Service providers **publish** their services to UDDI
- Service consumers **search and discover** available services
- Uses **XML and SOAP** for communication
- Fundamental component in **SOA (Service-Oriented Architecture)**

Example: UDDI in Action

Your company needs a payment processing web service. You search in a **UDDI registry**, find available payment services (like Razorpay or Stripe), and integrate the most suitable one into your application.

7.5 Quick Comparison Table

Protocol	Purpose	Example
SMS	Send short text messages	Bank OTP, alerts
HTTP	Load and display web pages	Opening any website
SMTP	Send emails	Sending Gmail
UDDI	Discover and find web services	Finding a payment API

★ Quick Tip / Memory Trick

Easy Memory Trick:
- **SMS** → Message on Mobile
- **HTTP** → Web Browser Pages
- **SMTP** → Email Sending
- **UDDI** → Service Directory / Yellow Pages

XML Parsing: DOM, SAX, and StAX

Why Do We Need XML Parsers?

XML (Extensible Markup Language) files store structured data. To **read, process, or modify** this data in an application, we need a parser. There are three main approaches: DOM, SAX, and StAX.

8.1 DOM — Document Object Model

DOM — Document Object Model

DOM loads the **entire XML file into memory** and creates a **tree structure** that represents every element in the document. Once loaded, you can navigate freely anywhere in the tree.

✓ Real-Life Analogy

Like **downloading a full movie** before watching. You wait for the whole thing to load, but then you can jump to any scene instantly.

⋆ Key Points to Remember

- Loads the **entire XML** into memory as a tree
- You can navigate **forward and backward** freely
- You can **modify** (add, delete, update) XML data
- **Easy** to use and understand
- **High memory usage** — not suitable for very large files
- **Slower** to start (must parse all data first)

Example: DOM in Action

XML file contains student: Name = Mausam, Branch = CSE.
DOM **loads the entire file**, builds a tree. You can then ask: "What is the student's name?" or "What is the branch?" — you can access any node at any time, in any order.

8.2 SAX — Simple API for XML

SAX — Simple API for XML

SAX reads the XML file **sequentially, line by line**, and fires **events** when it encounters specific elements (start tag, end tag, characters). Your code **reacts** to these events (push model).

✓ Real-Life Analogy

Like listening to **live radio**. Someone reads the story to you line by line. You cannot rewind or skip forward — you must listen in order.

⋆ Key Points to Remember

- **Event-driven, push model** — parser fires events, your handler reacts
- Reads **sequentially** — cannot go backward
- **Very fast** and **low memory** usage
- No random access — you can only process data as it appears
- Best for **large XML files** where only specific data is needed
- Cannot **modify** XML content

Example: SAX Events in Action

Reading student XML:

- Parser encounters `<student>` → fires "Start Element" event
- Parser reads "Mausam" → fires "Characters" event
- Parser encounters `</student>` → fires "End Element" event
- Your handler code processes each event as it comes

8.3 StAX — Streaming API for XML

StAX — Streaming API for XML

StAX is a **pull-based** streaming parser. Unlike SAX where the parser pushes events to your code, in StAX **your code pulls/requests** the next element when it is ready to process it. You are in control.

✓ Real-Life Analogy

Like watching **YouTube**. You control what you watch, when to **play**, **pause**, or **skip**. You are in charge, not the broadcaster.

⋆ Key Points to Remember

- **Pull model** — your code requests the next event/element
- **Faster than DOM** (no full tree in memory)
- **More control than SAX** (you decide when to read next)
- **Low memory** usage
- Can also be used to **write XML**
- Slightly more complex than SAX to implement

Example: StAX in Action

Your code says: "Give me the next XML event." Parser returns: start element `<name>`.
Your code processes it, then says: "Give me the next event." Parser returns: charac-

ters “Mausam”. You control the pace of reading completely.

8.4 DOM vs SAX vs StAX — Full Comparison

Feature	DOM	SAX	StAX
Style	Tree-based	Event (push)	Pull-based
Memory Usage	High	Low	Low
Speed	Slower	Fast	Fast
Navigation	Forward & Backward	Forward only	Controlled
Can Modify XML?	Yes	No	Yes (write)
Who Controls?	You (random access)	Parser (push events)	You (pull)
Best For	Small/medium XML	Large XML	Medium XML

★ Quick Tip / Memory Trick

Super Easy Memory Trick:
- **DOM** → **D**ownload whole XML (like downloading full movie)
- **SAX** → **S**ystem tells you (like live radio — you just listen)
- **StAX** → **St**ream and control (like YouTube — you play/pause)

Service-Oriented Architecture (SOA)

What is SOA?

Service-Oriented Architecture (SOA) is a software design approach where a system is built as a collection of **small, independent, reusable services** that communicate with each other over a network to form a complete application.

In one line: "Big system = collection of small independent services"

9.1 Core Concept

Instead of building one massive application, SOA splits it into **loosely coupled services** that each perform a specific business function. These services can be used and reused across different applications.

Example: Shopping Application

A shopping app built with SOA has separate, independent services:
- **Login Service** — handles authentication
- **Product Service** — manages product catalogue
- **Payment Service** — processes payments
- **Order Service** — manages orders
- **Delivery Service** — tracks shipments

Each works independently but together they form one complete shopping system.

9.2 The Three Key Entities of SOA

SOA has three fundamental actors/roles:

▶ 1. Service Provider

Service Provider

The **Service Provider** is the entity that **creates, owns, and provides** a web service. It implements the service logic and publishes it to the service registry so others can find and use it.

Example: Service Provider

A company that builds and hosts a **Payment Processing Service** (like Razorpay or Stripe). They offer the functionality "process this payment" and publish details about how to use it.

▶ 2. Service Consumer (Client)

Service Consumer

The **Service Consumer** (also called the *client*) is the entity that **uses/calls** a web service provided by the service provider to accomplish a task within its own application.

Example: Service Consumer

A **shopping app** that calls the Razorpay payment service when a user clicks "Pay Now." The shopping app is the consumer — it doesn't build the payment system itself, it just uses the service.

▶ 3. Service Registry (Broker)

Service Registry / Broker

The **Service Registry** is a centralised **directory or repository** where service providers **publish** their services and service consumers **search and discover** available services.

The standard technology used for SOA service registries is **UDDI**.

Example: Service Registry

An online **API marketplace** or **UDDI registry** where:
- Razorpay **publishes** its payment service with all technical details
- Your shopping app **searches** for a payment service
- The registry returns Razorpay's service details
- Your app **connects** to Razorpay and integrates it

9.3 How SOA Works — The Complete Flow

Step	Actor	Action
1	Service Provider	Builds the service and **publishes** it to the registry
2	Service Consumer	**Searches** the registry for a suitable service
3	Service Registry	Returns service details (location, interface, contract)
4	Service Consumer	**Binds** (connects) to the provider directly
5	Both	Service is **used** — provider executes, consumer receives result

✓ Real-Life Analogy

Restaurant Analogy:
- **Provider** = Restaurant (provides food / service)
- **Consumer** = Customer (uses the service)
- **Registry** = Food delivery app (lists all restaurants)
- Flow: Customer searches app → finds restaurant → orders food → eats

9.4 Key Features of SOA

⋆ Key Points to Remember

- **Loose Coupling:** Services are independent — changing one doesn't break others
- **Reusability:** One service (e.g., payment) can be used by many applications
- **Platform Independence:** Services communicate via standard protocols (SOAP, REST) regardless of the technology used to build them
- **Interoperability:** Java service can communicate with a .NET service seamlessly
- **Scalability:** Individual services can be scaled independently
- **Discoverability:** Services are found through the registry

9.5 Advantages and Disadvantages

Advantages:

✓ Easy to update one service without rewriting the whole application
✓ Reuse the same service across multiple applications
✓ Flexible and scalable architecture
✓ Platform and language independent

Disadvantages:

× Complex design and management
× Heavy network dependency
× Security challenges between services
× Performance overhead due to service calls

★ Quick Tip / Memory Trick

Quick Memory Trick for SOA Entities:
- **Provider** → *Gives* the service (creates and publishes)
- **Consumer** → *Uses* the service (finds and calls)
- **Registry** → *Lists* the services (like a directory or marketplace)

• Exam-Ready Definition

SOA (Service-Oriented Architecture) is a software design paradigm where an application is built as a collection of **loosely coupled, reusable services** that communicate over a network. It has three main entities: the **Service Provider** (creates services), the **Service Consumer** (uses services), and the **Service Registry/Broker** (helps discover services, typically using UDDI).

Master Revision Cheat Sheet

10.1 All Topics at a Glance

Topic	Core Idea	Key Example
SaaS	Use ready-made software	Gmail, Netflix
PaaS	Deploy your code, platform handles rest	Heroku, Firebase
IaaS	Rent virtual machines, control everything	AWS EC2, Azure VM
FaaS	Run functions on triggers, serverless	AWS Lambda
Virtualization	1 machine → many VMs via hypervisor	VirtualBox + Kali
Full Virt.	OS unaware, hypervisor fools it	VirtualBox
Para Virt.	OS modified to cooperate with hypervisor	Xen
Distributed Sys.	Many machines → 1 unified system	Google Drive
Cluster	Same location, high performance	Supercomputers
Grid	Different locations, resource sharing	SETI@home
Cloud	Services over internet, pay-as-you-go	AWS, Google Cloud
Community Cloud	Shared cloud for similar organisations	Hospitals, Govt
FTP	Transfer files (Port 21)	FileZilla
SNTP	Sync system clocks (Port 123)	Phone auto time
XaaS	Everything as a cloud service	AWS, Office 365
SMS	Short text via mobile network	OTP messages
HTTP	Web pages, request-response (Port 80)	Opening websites
SMTP	Send emails (Port 25/587)	Gmail sending
UDDI	Discover and register web services	Service registry
DOM	Full XML in memory as tree	Small XML files

★ **Quick Tip / Memory Trick**

Final Memory Tricks Summary:

Service Models: SaaS = Use | PaaS = Build | IaaS = Control | FaaS = Run Functions

Computing Types: Cluster = Same place | Grid = Different places | Cloud = Internet service

XML Parsers: DOM = Download all | SAX = System tells you | StAX = You control

SOA Roles: Provider = Gives | Consumer = Uses | Registry = Lists